CENTRO UNIVERSITÁRIO SENAC-RS

UNISENAC • CAMPUS PELOTAS

TRABALHO PRÁTICO

EVOLUÇÃO DE UM AGENTE INTELIGENTE NO ATARI

Neuroevolução aplicada ao Freeway

> **OBJETIVO CENTRAL**
> Construir um agente que melhore ao longo de várias gerações utilizando população, avaliação, fitness, seleção, elitismo e mutação. O foco é compreender e demonstrar o processo evolutivo - não obter um agente perfeito.

CURSO	Análise e Desenvolvimento de Sistemas
UNIDADE CURRICULAR	Fundamentos de Inteligência Artificial
PROFESSOR	**Pablo De Chiaro Rosa**
PERÍODO	2026/2

A lógica do trabalho:

POPULAÇÃO → AVALIAÇÃO → FITNESS → SELEÇÃO → MUTAÇÃO → NOVA GERAÇÃO

1 O problema que será resolvido

Vocês deverão desenvolver um agente capaz de tomar decisões dentro do jogo Freeway, da Arcade Learning Environment (ALE). O agente não receberá exemplos de respostas corretas e não utilizará DQN, Q-Learning ou backpropagation para aprender.

A estratégia será evolutiva: vários agentes diferentes serão testados; os de melhor desempenho serão selecionados; seus parâmetros serão copiados e modificados; e esse ciclo será repetido por várias gerações.

> **IMPORTANTE**
> O código inicial fornece a comunicação com a rede neural e a estrutura geral. As etapas evolutivas marcadas como TODO fazem parte do trabalho e deverão ser implementadas, testadas e explicadas pelo grupo.

Freeway como ambiente-base

Item	Definição no trabalho
Ambiente	ALE/Freeway-v5
Observação	RAM do Atari: vetor com 128 valores
Ações	0 = NOOP, 1 = UP, 2 = DOWN
Objetivo	Atravessar as faixas de tráfego e alcançar o topo da tela
Recompensa	O ambiente fornece recompensa quando a travessia é concluída

Antes de programar

O grupo deverá estudar a documentação oficial e conseguir demonstrar, no próprio código, que compreendeu:

- como criar e reiniciar o ambiente;
- qual é o formato do estado retornado pelo Freeway;
- como consultar o espaço de ações;
- como executar uma ação com env.step(...);
- o significado de reward, terminated e truncated;
- quando um episódio termina e como acumular a recompensa.

> **REGRA DE IMPLEMENTAÇÃO**
> Não basta copiar o template. Cada grupo deverá conseguir apontar no código onde o estado é recebido,

onde a ação é escolhida, onde a recompensa é obtida e onde o episódio é encerrado.

2 Conceitos obrigatórios: o que deve existir no programa

Conceito	Significado	O que o grupo deve implementar
Agente	Elemento que toma decisões.	Criar uma estrutura que receba o estado e retorne uma ação.
Ambiente	O jogo com o qual o agente interage.	Criar, reiniciar e executar o Freeway via Gymnasium/ALE.
Ação	Decisão tomada pelo agente.	Escolher entre NOOP, UP e DOWN a partir da saída da rede.
Recompensa	Sinal retornado pelo ambiente após uma ação.	Acumular os rewards recebidos durante o episódio.
Fitness	Critério usado para comparar indivíduos.	Transformar o desempenho do episódio em um valor usado pela seleção.
Indivíduo	Uma possível solução.	Cada agente deve possuir seus próprios pesos/parâmetros.
População	Conjunto de indivíduos da mesma geração.	Criar vários agentes diferentes e avaliar todos.
Geração	Um ciclo completo da evolução.	Avaliar a população, selecionar e produzir a próxima população.
Seleção	Escolha dos indivíduos que terão descendentes.	Dar preferência aos maiores fitness conforme uma estratégia definida pelo grupo.
Elitismo	Preservação direta das melhores soluções.	Copiar pelo menos o melhor indivíduo sem sofrer mutação.
Mutação	Alteração aleatória do DNA/pesos.	Modificar parte dos pesos usando taxa e intensidade de mutação.
Exploração	Busca por soluções diferentes.	Gerar diversidade com população inicial aleatória e mutações.
Convergência	Estabilização do processo evolutivo.	Observar se melhor e média do fitness deixam de melhorar por várias gerações.

Conceito	Significado	O que o grupo deve implementar

RECOMPENSA ≠ FITNESS
Recompensa é o sinal entregue pelo ambiente durante a interação. Fitness é a medida usada pelo algoritmo evolutivo para comparar indivíduos. Neste trabalho, o fitness pode começar como a soma das recompensas do episódio - mas o grupo deve saber explicar a diferença.

3 Fluxo obrigatório da evolução

#	Etapa	Resultado esperado
1	Criar população inicial	Gerar vários agentes com pesos diferentes.
2	Avaliar indivíduos	Cada agente executa um episódio no Freeway.
3	Calcular fitness	Registrar o desempenho de cada indivíduo.
4	Selecionar	Escolher indivíduos de melhor desempenho para reprodução.
5	Aplicar elitismo	Preservar diretamente o(s) melhor(es).
6	Reproduzir	Criar filhos a partir dos indivíduos selecionados.
7	Mutar	Modificar aleatoriamente parte dos pesos dos filhos.
8	Formar nova geração	Completar novamente o tamanho definido da população.
9	Repetir	Executar o ciclo durante várias gerações.

Representação do indivíduo

Cada indivíduo será uma pequena rede neural. Os pesos e bias dessa rede funcionam como o "DNA" evoluído pelo algoritmo.

```
128 valores da RAM  →  16 neurônios  →  3 saídas  →  NOOP / UP / DOWN
```

O template já fornece essa política básica. O objetivo do grupo não é treinar a rede por gradiente; é evoluir seus parâmetros por seleção e mutação.

Parâmetros iniciais sugeridos

Parâmetro	Valor inicial	Pode ser alterado?
Tamanho da população	20	Sim, justificando
Número de gerações	30	Sim, justificando
Elite	2 indivíduos	Sim, mantendo pelo menos 1
Taxa de mutação	10%	Sim - parte do experimento
Intensidade da mutação	0,20	Sim - parte do experimento

Nesta versão, não é obrigatório implementar crossover: a reprodução pode ser feita por cópia do pai seguida de mutação.

4 Template inicial em Python - Parte 1

Este código é o ponto de partida. As funções marcadas com TODO deverão ser implementadas pelo grupo. O método agir(...) é fornecido para manter o foco da atividade no processo evolutivo.

Instalação mínima: pip install "gymnasium[atari]" numpy matplotlib

```
import copy
import gymnasium as gym
import ale_py
import numpy as np

gym.register_envs(ale_py)

# ------------------------------------------------------------
# CONFIGURAÇÕES
# ------------------------------------------------------------

ENV_ID = "ALE/Freeway-v5"

TAMANHO_POPULACAO = 20
NUM_GERACOES = 30
NUM_ENTRADAS = 128
NUM_OCULTOS = 16
NUM_ACOES = 3

TAXA_MUTACAO = 0.10
INTENSIDADE_MUTACAO = 0.20
NUM_ELITE = 2


def criar_ambiente(render_mode=None):
    return gym.make(
        ENV_ID,
        obs_type="ram",
        render_mode=render_mode
    )


class Agente:
    def __init__(self):
        # Cada indivíduo começa com parâmetros diferentes.
        self.W1 = np.random.randn(NUM_ENTRADAS, NUM_OCULTOS)
        self.b1 = np.random.randn(NUM_OCULTOS)
        self.W2 = np.random.randn(NUM_OCULTOS, NUM_ACOES)
        self.b2 = np.random.randn(NUM_ACOES)
        self.fitness = 0.0
```

```
    def agir(self, estado):
        # RAM: valores inteiros entre 0 e 255.
        estado = estado.astype(np.float32) / 255.0

        oculto = np.tanh(estado @ self.W1 + self.b1)
        saida = oculto @ self.W2 + self.b2

        # A maior saída define a ação escolhida.
        return int(np.argmax(saida))
```

O QUE VOCÊ PRECISA ENTENDER AQUI

Por que existem 128 entradas? Por que a saída possui 3 valores? O que significa escolher argmax? Onde estão armazenados os parâmetros que serão evoluídos?

5 Template inicial em Python - Parte 2

```
def avaliar(agente):
    """
    TODO - AGENTE + AMBIENTE + AÇÃO + RECOMPENSA + FITNESS

    1. Criar/reiniciar o ambiente.
    2. Receber o estado inicial.
    3. Enquanto o episódio não terminar:
       a) pedir uma ação ao agente;
       b) executar a ação no ambiente;
       c) receber novo estado e reward;
       d) acumular a recompensa.
    4. Fechar o ambiente.
    5. Retornar o fitness do indivíduo.
    """
    pass


def criar_populacao():
    """
    TODO - INDIVÍDUO + POPULAÇÃO

    Criar TAMANHO_POPULACAO agentes independentes.
    Todos devem começar com parâmetros aleatórios próprios.
    """
    pass


def selecionar(populacao):
    """
    TODO - SELEÇÃO

    Escolher os indivíduos que poderão gerar descendentes.
    O grupo deverá definir e explicar seu critério.
    """
    pass


def mutar(agente):
    """
    TODO - MUTAÇÃO + EXPLORAÇÃO

    Alterar aleatoriamente parte de W1, b1, W2 e b2.
    Utilizar obrigatoriamente:
        TAXA_MUTACAO
        INTENSIDADE_MUTACAO
    """
    pass


def nova_geracao(populacao):
    """
    TODO - ELITISMO + REPRODUÇÃO + MUTAÇÃO

    1. Selecionar os pais.
    2. Copiar NUM_ELITE melhores sem alteração.
    3. Criar descendentes a partir dos selecionados.
    4. Aplicar mutação nos descendentes.
    5. Retornar exatamente TAMANHO_POPULACAO agentes.
    """
```

```
    pass
```

REPRODUÇÃO
Para esta atividade, um filho pode começar como uma cópia independente de um pai (por exemplo, com copy.deepcopy) e depois sofrer mutação. O grupo deverá garantir que alterar o filho não altere o objeto do pai.

6 Template inicial em Python - Parte 3

```
populacao = criar_populacao()
historico = []

for geracao in range(NUM_GERACOES):

    # Avaliar TODOS os indivíduos da geração.
    for agente in populacao:
        agente.fitness = avaliar(agente)

    valores = [agente.fitness for agente in populacao]

    melhor = max(valores)
    media = float(np.mean(valores))
    pior = min(valores)

    historico.append({
        "geracao": geracao,
        "melhor": melhor,
        "media": media,
        "pior": pior,
    })

    print(
        f"Geração {geracao:03d} | "
        f"Melhor: {melhor:.2f} | "
        f"Média: {media:.2f} | "
        f"Pior: {pior:.2f}"
    )

    # Produzir a próxima geração.
    populacao = nova_geracao(populacao)

# TODO:
# salvar o histórico em CSV
# gerar o gráfico de evolução
# executar/renderizar o melhor agente encontrado
```

Checklist de implementação

Componente	Critério mínimo
avaliar(agente)	Executa um episódio, acumula reward e retorna o fitness.
criar_populacao()	Produz vários indivíduos diferentes.
selecionar(populacao)	Escolhe os pais com base no fitness.
elitismo	Mantém pelo menos o melhor indivíduo sem mutação.

Componente	Critério mínimo
mutar(agente)	Altera parâmetros usando taxa e intensidade.
nova_geracao(populacao)	Combina seleção, elitismo, reprodução e mutação.
histórico	Registra melhor, média e pior fitness por geração.
gráfico	Apresenta a evolução do melhor fitness e do fitness médio.

7 Experimentos obrigatórios

Depois que o ciclo evolutivo estiver funcionando, o grupo deverá investigar o efeito da mutação. Não basta executar três vezes: é necessário comparar e interpretar os resultados.

Experimento	Taxa de mutação	Intensidade	Hipótese/observação
A	1%	0,05	Pouca alteração entre pais e filhos
B	10%	0,20	Configuração intermediária
C	40%	0,50	Grande diversidade e maior risco de destruir boas soluções

Exploração

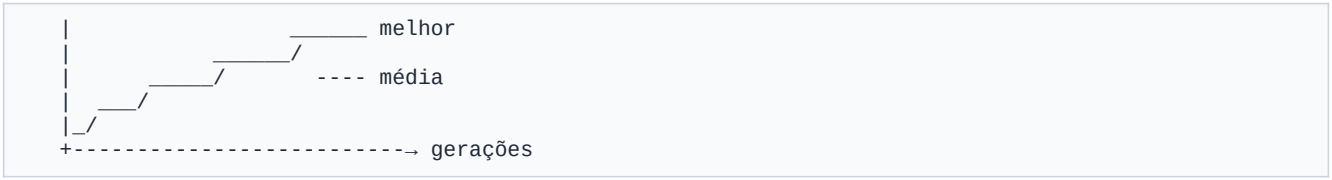
Vocês deverão explicar de onde surge a exploração no algoritmo. Neste projeto, ela acontece principalmente pela população inicial aleatória e pelas mutações que introduzem novas soluções.

> **PERGUNTA DE ANÁLISE**
> O que acontece se a mutação for quase zero? E se cada geração alterar agressivamente quase todos os pesos?

Convergência

Registrem o melhor fitness e o fitness médio de todas as gerações. A partir desses dados, analisem se o processo continuou melhorando ou se estabilizou.

```
fitness
   ↑
```



Comparação mínima

Antes da evolução, executem também um agente que escolha ações aleatoriamente. Comparem seu desempenho com o melhor indivíduo evoluído. Essa comparação funciona como baseline simples para verificar se o processo encontrou comportamento melhor do que decisões sem estratégia.

8 Entrega e defesa do trabalho

Estrutura mínima da entrega

```
projeto/
├── main.py
├── README.md
└── resultados/
    ├── fitness.csv
    └── fitness.png
```

O README.md deverá registrar a configuração usada no experimento e permitir que outra pessoa execute o projeto sem alterar manualmente o código.

- ambiente utilizado e forma de observação;
- tamanho da população e número de gerações;
- estratégia de seleção;
- quantidade de indivíduos preservados por elitismo;
- taxa e intensidade de mutação;
- resultado dos três experimentos;
- gráfico com melhor fitness e fitness médio;
- análise sobre exploração e convergência.

Perguntas que o grupo deverá saber responder

1. O que é o agente e qual estado ele recebe do ambiente?
2. Qual a diferença entre ação, recompensa e fitness?
3. Como indivíduo, população e geração aparecem no código?
4. Como a seleção foi implementada e por que esse critério foi escolhido?
5. O que aconteceria se não existisse elitismo?
6. Como taxa e intensidade de mutação afetam os descendentes?
7. Onde ocorre exploração no algoritmo?
8. Os resultados indicam convergência? Qual evidência sustenta a resposta?
9. O melhor agente superou o agente aleatório? Qual evidência mostra isso?
10. Qual mudança vocês fariam para tentar melhorar o resultado?

Critério central: relacionem cada conceito ao trecho correspondente do código e às evidências produzidas nos experimentos.

Documentação oficial: Freeway • Interface Gymnasium/ALE